Software Engineering

BCSE301L

Module 4

Dr. Naveenkumar Jayakumar
Dept. of Computational Intelligence
PRP-217-4

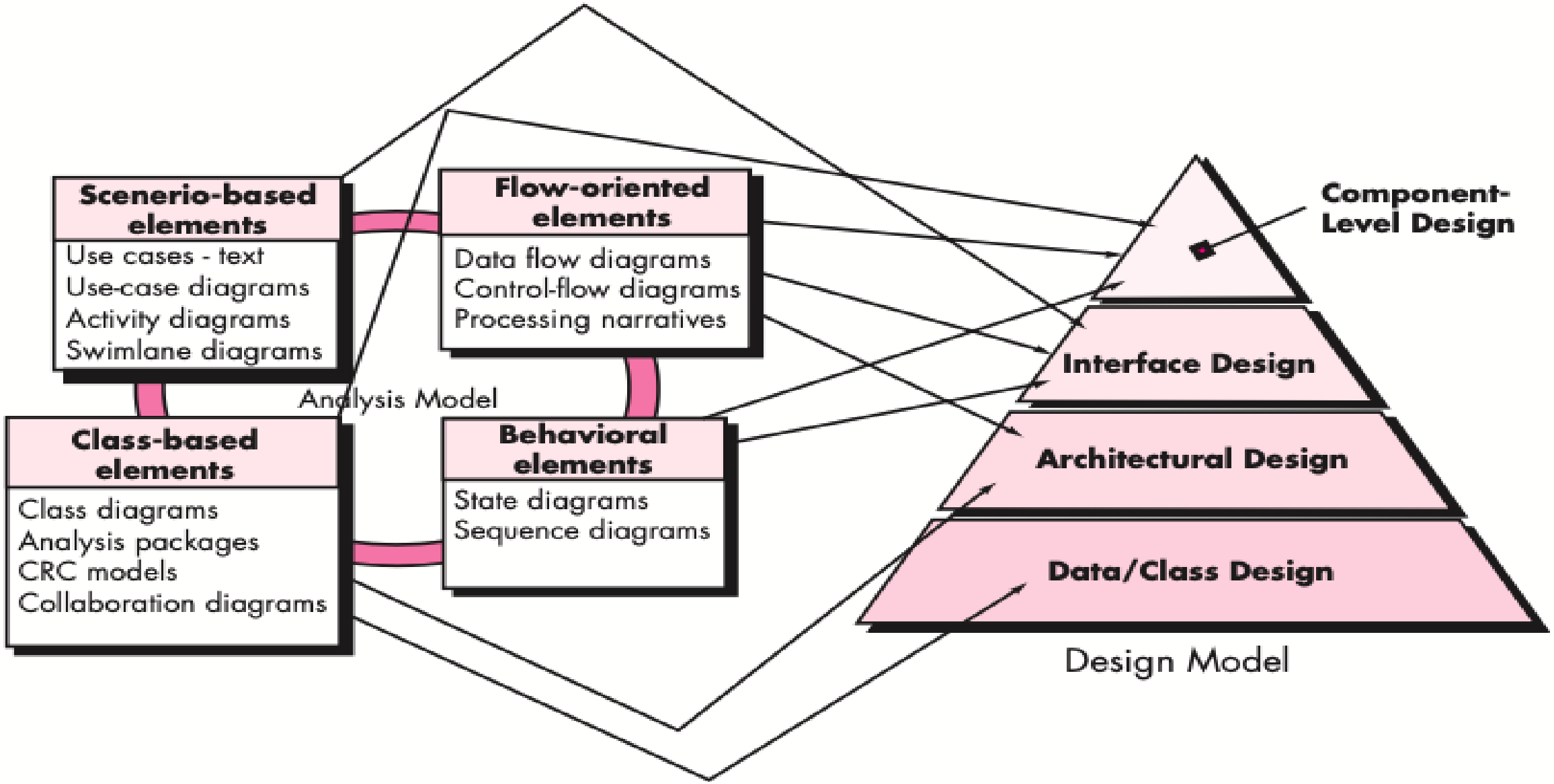
Design Concepts

Software Design and Its Role	Core part of software engineering
	Applied regardless of the software process model.
	Begins after software requirements are analysed and modelled.
	The final step in modeling before construction (code generation & testing).

Design Concepts

Design Models & Flow of Information	Uses elements from the requirements model (scenario-based, class-based, flow-oriented, and behavioural).
	Produces four essential design models
	Data/Class Design
	Architectural Design
	Interface Design
	Component Level Design

Translating Requirement model into Design Model



Translating Requirement model into Design Model

Requirement Analysis Components	Design Models
Scenario-based elements: Represent user interactions and system behavior	**Data/Class Design**: Defines the data structures and class relationships
Flow-oriented elements: Describe the flow of data and control in the system	**Architectural Design**: Establishes the overall system structure, including layers, modules, and components.
Class-based elements: Define the system's structure	**Interface Design**: Focuses on user interfaces (UI), APIs, and external system interactions.
Behavioral elements: Model dynamic behavior	**Component-Level Design**: Defines the internal implementation details of individual components/modules.

Translating Requirement model into Design Model

Data/Class Design

- Converts class models into design class realizations.
- Defines data structures needed for implementation.
- Uses CRC diagrams and class attributes for data design.
- Some class design integrates with software architecture design.

Architectural Design

- Defines relationships between major structural elements.
- Uses architectural styles and design patterns.
- Considers constraints on architecture implementation.
- Derived from the requirements model.

Translating Requirement model into Design Model

Interface Design

- Specifies how software interacts with other systems and users.
- Defines information flow (data/control) and behaviour.
- Based on usage scenarios and behavioural models.

Component Level Design

- Converts software architecture elements into procedural descriptions.
- Uses class-based, flow, and behavioural models for component definition.

Importance of Software Design

Importance of Software Design	Directly impacts software construction and maintainability.
	Ensures quality in software engineering.
	Provides representations that can be assessed for quality.
	Serves as the foundation for all subsequent software engineering activities.
	Accurately translates stakeholder requirements into a functional system.
	Prevents the risk of an unstable, hard-to-test, and unassessable system.

Software Design Overview

- Software design is an iterative process that translates requirements into a blueprint for software development.

- It starts with a high-level overview and gradually becomes more detailed.

- Each refinement ensures the design traces back to requirements while increasing specificity.

Software Quality Guidelines & Attributes

- The design is reviewed through technical reviews to ensure quality.

- Three key characteristics of good design:
  - It should implement all explicit and implicit requirements.
  - It should be readable and understandable for developers and testers.
  - It should provide a complete picture of software, including data, functionality, and behavior.

Design Principles

Avoid ‘Tunnel Vision’

- Don’t focus too much on one aspect of design and ignore others.
- Keep a broad perspective and consider all requirements.

Traceable to the Analysis Model

- The design should directly connect to the requirements and analysis models.
- Every feature in the design should have a clear reason based on earlier analysis.

Design Principles

Don’t Reinvent the Wheel

- Use existing solutions, libraries, and design patterns instead of creating everything from scratch.
- Saves time and ensures proven, reliable implementations.

Minimize Intellectual Distance

- The software design should be as close as possible to how the problem exists in the real world.
- Makes the system easier to understand, modify, and maintain.

Design Principles

Uniformity & Integration

- The design should have a consistent structure, style, and logic across all components.
- Avoid different teams working in disconnected ways.

Accommodate Change

- Software should be designed to easily adapt to new features or requirements.
- Use modular and scalable approaches to allow flexibility.

Design Principles

Graceful Degradation

- Even if something goes wrong (bad data, errors, unexpected conditions), the system should not crash completely.
- Implement error handling and fallback mechanisms.

Design is NOT Coding; Coding is NOT Design

- Design focuses on planning and structure, while coding is the actual implementation.
- Good design guides coding but is separate from it.

Design Principles

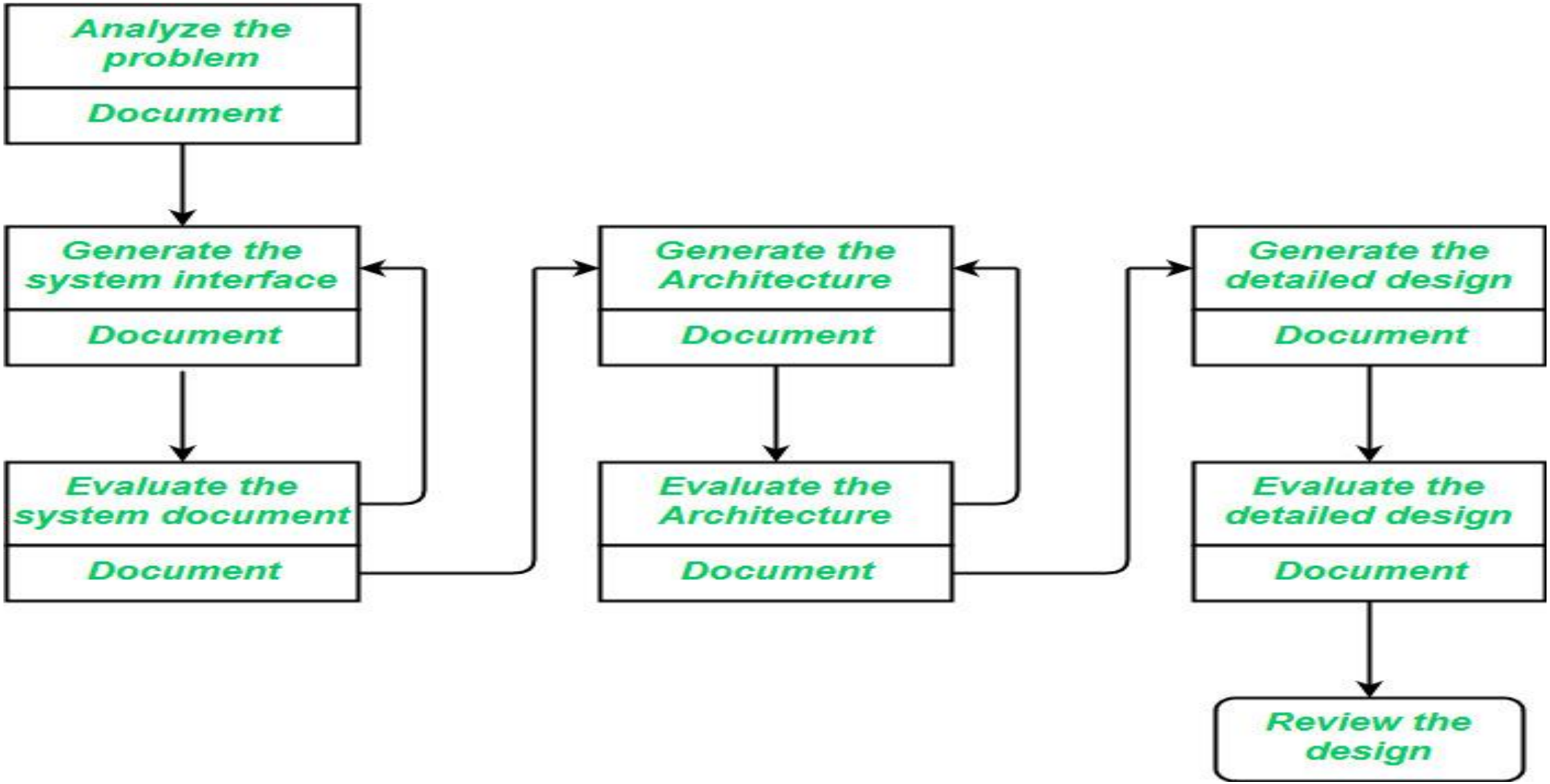
Assess Quality During Design

- Design should be evaluated for efficiency, scalability, and correctness while it is being created.
- Avoid waiting until the end of development to check for quality issues.

Review to Minimize Conceptual Errors

- Regular peer reviews and feedback should be used to catch logical flaws early.
- Helps prevent misunderstandings and incorrect assumptions in the design.

Design Process



Design Concepts Overview

- These concepts help in creating better designs and applying more advanced design methods.
- They guide software designers in answering important questions:
  - How to divide software into smaller components?
  - How to separate details of functions and data structures from the overall software concept?
  - What standards define a high-quality software design?

Design Concepts – 1. Abstraction

- Abstraction helps in breaking down a problem into different levels.
- Abstraction is the process of hiding implementation details and showing only essential features of a software system.
- It helps in reducing complexity by allowing developers to focus on what a system does rather than how it is implemented.

Design Concepts – 1. Abstraction

Abstraction Types	Definition	Example
Procedural Abstraction	It refers to breaking down a process into well-defined procedures (functions or methods) that perform specific tasks without exposing the internal implementation.	A function calculateInterest(principal, rate, time) that computes interest without revealing the mathematical formula inside.
Data Abstraction	It refers to defining data structures in a way that hides the implementation details and only exposes necessary functionalities.	A Car class with attributes like speed, color, and methods like startEngine() without exposing internal engine mechanics.
Control Abstraction	Control abstraction hides the details of control flow and execution logic, allowing developers to focus on high-level tasks rather than low-level control mechanisms.	Loops & Iterators Instead of manually managing loop counters and conditions, we use constructs like for-each loops

Design Concepts – 1. Abstraction - Data

```
#include <iostream>
using namespace std;

class Calculator {
private:
   int result;  // Private attribute (hidden from direct access)

public:
   Calculator() : result(0) {}  // Constructor initializes result to 0

   int getResult() const {
      return result;  // Public method to access the private result
   }

   void add(int a, int b);
   void subtract(int a, int b);
};
```

The result variable is private, meaning it cannot be accessed directly from outside the class. The user can only access the result through the getResult() method, which abstracts away the internal data structure.

Design Concepts – 1. Abstraction - Procedural

```
void Calculator::add(int a, int b) {
    result = a + b;  // Encapsulates the addition logic
}

void Calculator::subtract(int a, int b) {
    result = a - b;  // Encapsulates the subtraction logic
}
```

The add() and subtract() methods encapsulate the logic for performing addition and subtraction. The user doesn't need to know how the calculation is done; they just call these methods to perform the operations.

Design Concepts – 1. Abstraction - Control

```
void runCalculator() {
    Calculator calc;
    int choice, a, b;
    while (true) {
        cout << "\n1. Add\n2. Subtract\n3. Get Result\n4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1:
                cout << "Enter first number: ";
                cin >> a;
                cout << "Enter second number: ";
                cin >> b;
                calc.add(a, b);
                break;
            case 2:
                cout << "Enter first number: ";
                cin >> a;
                cout << "Enter second number: ";
                cin >> b;
                calc.subtract(a, b);
                break;
            case 3:
                cout << "Result: " << calc.getResult() << endl;
                break;
            case 4:
                cout << "Exiting...\n";
                return;
            default:
                cout << "Invalid choice. Please try again.\n";
        }
    }
}
```

The runCalculator() function abstracts the control flow of the program.

It provides a simple menu-driven interface that allows the user to interact with the calculator without worrying about the underlying control logic (e.g., loops, conditionals).

Design Concepts – 2. Architecture

- Software architecture defines the structure of software components and their interactions.

- It serves as a framework for detailed design activities.

- Architectural patterns help solve common design problems.

Design Concepts – 2. Architecture

- **Key properties of architectural design**
  - **Structural properties** – Defines system components (e.g., modules, objects) and their interactions.
  - **Extra-functional properties** – Addresses performance, security, reliability, adaptability, etc.
  - **Families of related systems** – Uses repeatable design patterns for similar systems.

Design Concepts – 2. Architecture

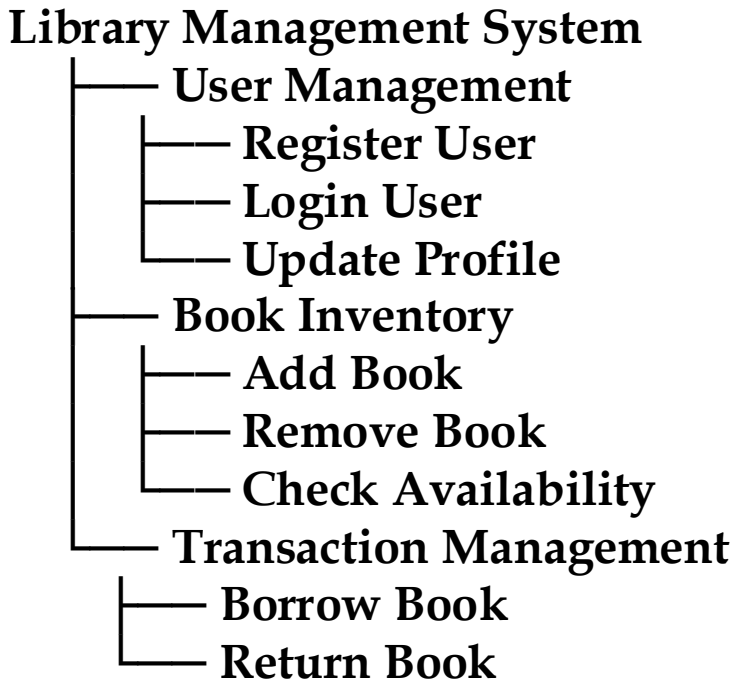
Architecture Model Types	Definition	Example
Structured Model	Represents software architecture as a structured collection of program components (e.g., modules, objects, filters) and their interactions.	A Layered Architecture (e.g., in web applications: Presentation Layer → Business Logic Layer → Data Access Layer).
Framework Model	Increases abstraction by identifying repeatable architectural design frameworks used in similar applications.	framework in web development, where the Model manages data, the View handles UI, and the Controller processes input.
Dynamic Model	Focuses on the behavioral aspects of architecture, showing how the system structure or configuration changes due to external events.	A real-time traffic monitoring system that dynamically adjusts signals based on real-time traffic data

Design Concepts – 2. Architecture

Architecture Model Types	Definition	Example
Process Model	Represents the business or technical processes that the system must accommodate, focusing on how different tasks are executed.	An Order Processing System, where an online order goes through steps like Order Placement → Payment Processing → Inventory Check → Shipping.
Functional Model	Represents the functional hierarchy of the system, showing how different functions interact and depend on each other.	A Banking System, where high-level functions like "Manage Accounts" are broken down into sub-functions like "Create Account," "Deposit Money," and "Withdraw Money."

Design Concepts – 2. Architecture – Structured

- **Library Management System**
- The system is divided into several modules such as **User Management , Book Inventory , and Transaction Management** .
- Each module has specific responsibilities:
  - **User Management** : Handles user registration, login, and profile updates.
  - **Book Inventory** : Manages the list of books, their availability, and details.
  - **Transaction Management** : Handles borrowing and returning books.



Design Concepts – 2. Architecture – Framework

- **Library Management System**
- A Web Application Framework like Django (Python) or Spring Boot (Java) could be used to build the Library Management System.
- The framework provides pre-built components for handling common tasks such as:
  - Authentication : User login and registration.
  - Database Access : Managing book inventory and user data.
  - Routing : Handling requests for borrowing or returning books.

Library Management System Built Using Django Framework

- **Authentication (Built-in)**
- **Database ORM (Object-Relational Mapping)**
- **URL Routing (Request Handling)**
- **Templates (HTML Views)**

Design Concepts – 2. Architecture – Dynamic

- **Library Management System**

Sequence Diagram: Borrowing a Book

User -> System: Login
System -> Book Inventory: Check Availability
Book Inventory --> System: Book Available
System -> Transaction Management: Record Borrowing
Transaction Management --> System: Transaction Recorded
System --> User: Confirm Borrowing

Design Concepts – 2. Architecture – Dynamic

- **Library Management System**

Data Flow Diagram: Borrowing a Book

[User] -> (Request Borrow) -> [System]
[System] -> (Check Availability) -> [Book Inventory]
[Book Inventory] -> (Update Status) -> [Transaction Management]
[Transaction Management] -> (Confirm Borrow) -> [User]

Design Concepts – 2. Architecture – Functional

➢ Library Management System

- The functional requirements of the system include:
  - **Register User** : Allows new users to create an account.
  - **Login User** : Allows existing users to log in.
  - **Search Books** : Allows users to search for books by title, author, or genre.
  - **Borrow Book** : Allows users to borrow a book if it's available.
  - **Return Book** : Allows users to return a borrowed book.

➢It describes what the system does without worrying about how it does it.

Functional Requirements:

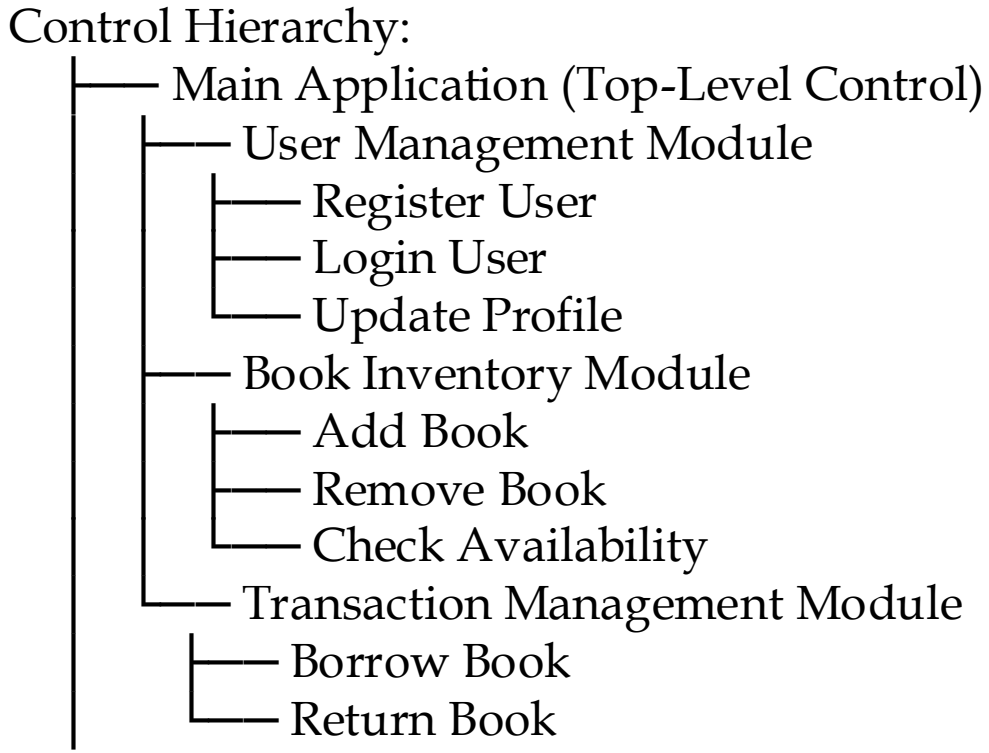
- **Register User**
- **Login User**
- **Search Books**
- **Borrow Book**
- **Return Book**

Design Concepts – 2. Architecture Concepts

Architecture Concept	Definition	Measures
Control Hierachy (Program Structure)	Defines how program components (modules) are organized. Control Hierarchy refers to the organization of control structures within a system, where higher-level components manage and delegate tasks to lower-level components. It defines how control flows from one module to another.	▪ Depth = Number of control levels in hierarchy. ▪ Width = Overall span of control (how many modules exist at each level). ▪ Fan-out: Number of modules directly controlled by another module. ▪ Fan-in: Number of modules that control a given module. ▪ Superordinate: A module that controls another module. ▪ Subordinate: A module that is controlled by another.

Design Concepts – 2. Architecture – Control Hierarchy

- The Main Application is responsible for orchestrating the overall flow of the system.
- It delegates specific tasks to lower-level modules like User Management , Book Inventory , and Transaction Management .
- For example, when a user wants to borrow a book, the Main Application calls the Transaction Management Module , which in turn interacts with the Book Inventory Module to check availability.



Design Concepts – 2. Architecture Concepts

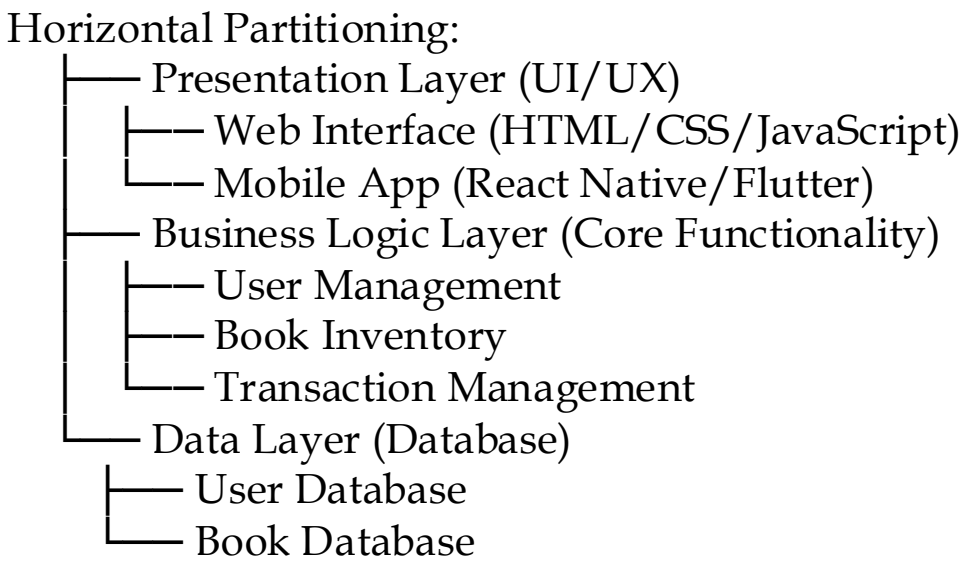
Architecture Concept	Definition	Merits and Demerits
Structural Partitioning		

Horizontal Partitioning	▪ Splits the software into separate functional branches in the hierarchy.
	▪ Divides into three main partitions:
	  ▪ Input (data collection & preprocessing).
	  ▪ Process (core computations & logic).
	  ▪ Output (results presentation & response).
	  ▪ It involves dividing the system into layers or tiers based on functionality. Each layer is responsible for a specific aspect of the system, such as presentation, business logic, or data storage.
		▪ **Merits**
		▪ Easier to test and debug.
		▪ Easier to maintain and modify.
		▪ Reduces side effects (errors don't spread easily).
		▪ Makes the software extensible (easy to add features).
		
		▪ **Demerits**
		▪ More data must be passed between modules.
		▪ Increases complexity in program flow control.

Design Concepts – 2. Architecture – Horizontal Partitioning

- The Presentation Layer handles user input and displays results (e.g., showing available books).
- The Business Logic Layer contains the core logic of the system (e.g., checking if a book is available before allowing it to be borrowed).
- The Data Layer stores and retrieves data from databases (e.g., user profiles, book inventory).

- Horizontal Partitioning allows for separation of concerns. Each layer has a specific responsibility, making the system more modular and easier to develop, test, and maintain. You can change the UI without affecting the business logic or data storage.



Design Concepts – 2. Architecture Concepts

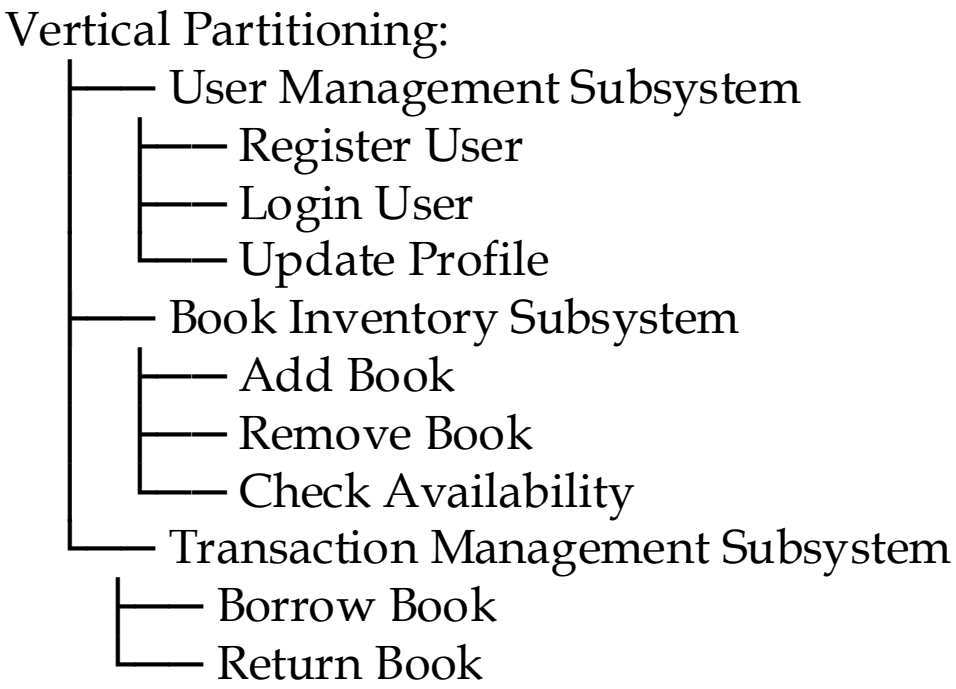
Architecture Concept	Definition	Effects
Structural Partitioning		
Vertical Partitioning (Factoring)	▪ Top-down structure: Control and decision-making are done at the top levels. ▪ High-level modules → Handle control functions (decision making). ▪ Low-level modules → Handle execution (input, computation, output).	▪ **Change in a control module** → Affects all its subordinate modules ▪ . ▪ **Change in a worker module** → Has minimal impact on other modules.

Design Concepts – 2. Architecture Concepts

Architecture Concept	Definition	Effects
	Structural Partitioning	
Vertical Partitioning (Factoring)	▪ Top-down structure: Control and decision-making are done at the top levels. ▪ High-level modules → Handle control functions (decision making). ▪ Low-level modules → Handle execution (input, computation, output). ▪ It involves dividing the system into independent functional units or subsystems. ▪ Each subsystem handles a specific part of the system's functionality, such as user management, book inventory, or transaction management.	▪ **Change in a control module** → Affects all its subordinate modules ▪ **Change in a worker module** → Has minimal impact on other modules.

Design Concepts – 2. Architecture – Vertical Partitioning

- Each subsystem is independent and focuses on a specific aspect of the system.
- For example, the User Management Subsystem is responsible for handling all user-related tasks, while the Book Inventory Subsystem manages book-related tasks.
- These subsystems can communicate with each other through well-defined interfaces, but they are otherwise decoupled.
- Allows for better scalability and modularity

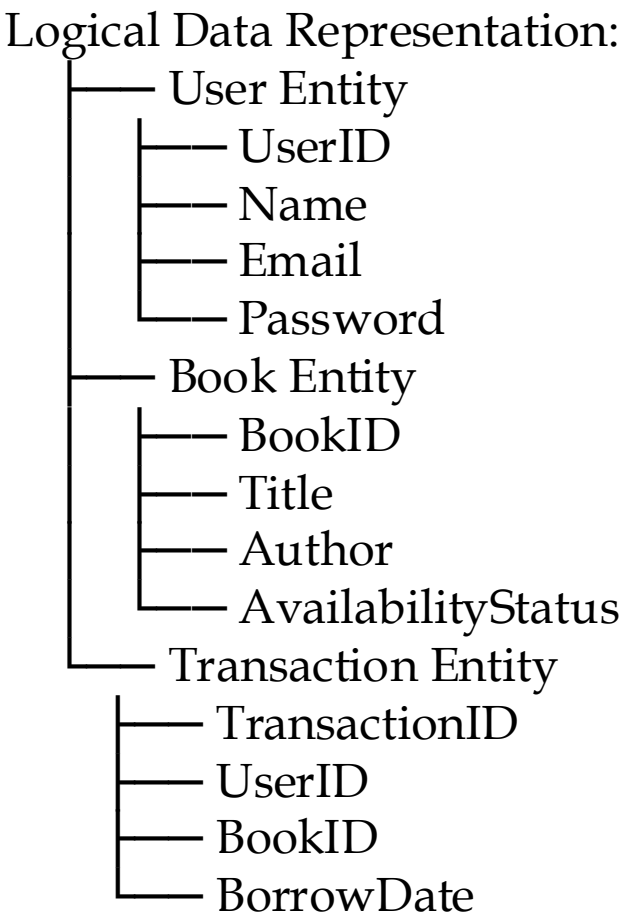


Design Concepts – 2. Architecture Concepts

Architecture Concept	Definition	Specifies
Logical Data Representation	▪ Defines how data elements relate to each other ▪ Refers to how data is structured and organized within the system. ▪ It focuses on the logical relationships between data entities rather than the physical storage details.	▪ **Specifies:** ▪ Data organization (how data is structured). ▪ Methods of access (how data is retrieved). ▪ Associations (how data points are connected). ▪ Processing alternatives (different ways to handle data).

Design Concepts – 2. Architecture - Logical Data Representation

- The Logical Data Representation defines how data is structured and related within the system.
- For example, a Transaction links a User to a Book , indicating that a user has borrowed a specific book.
- This logical representation is independent of how the data is physically stored (e.g., in a relational database, NoSQL database, or flat files).
- helps ensure that the system's data is well-organized and consistent.



Design Concepts – 3. Design Patterns

- A design pattern is a proven solution to a recurring problem in software design.
- It provides a named structure that can be reused across different projects.
- Patterns help designers decide:
  - Is the pattern suitable for the current problem?
  - Can it be reused to save design effort?
  - Can it guide the development of a similar but different pattern?

Design Concepts – 3. Design Patterns

Design Concepts – 4. Separation of Concerns

- Breaking down a complex problem into smaller, manageable parts makes it easier to solve.
- A concern refers to a specific feature or behavior in software.

- **Why is it useful?**
  - Solving one big problem is harder than solving many smaller problems.
  - Divide and conquer approach helps in software modularity.

Design Concepts – 5. Modularity

- Modularity is the practice of breaking down a system into smaller, manageable, and independent components (modules).

- These modules are designed to work independently but can be combined to form a complete system.

- Software architecture and design patterns follow modularity principles.

Design Concepts – 5. Modularity – Calculator Example

```
def main():
    while True:
        print("\n1. Add")
        print("2. Subtract")
        print("3. Multiply")
        print("4. Divide")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == '5':
            print("Exiting...")
            break

        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))

        if choice == '1':
            result = add(num1, num2)
            print(f"Result: {result}")
        elif choice == '2':
            result = subtract(num1, num2)
            print(f"Result: {result}")
        elif choice == '3':
            result = multiply(num1, num2)
            print(f"Result: {result}")
        elif choice == '4':
            result = divide(num1, num2)
            print(f"Result: {result}")
        else:
            print("Invalid choice. Please try again.")
```

Main Module

The **main module** acts as the entry point of the program. It coordinates the interaction between the user and the calculator's functionality. It delegates tasks to other modules based on user input.

- The main() function serves as the control center.
- It doesn't contain the logic for performing calculations; instead, it calls separate functions (add, subtract, etc.) to handle those tasks.
- This separation ensures that the main module remains focused on managing user interaction.

Design Concepts – 5. Modularity – Calculator Example

```
def add(a, b):
    return a + b
```

The add() function is a standalone module that performs addition. It takes two numbers as input and returns their sum. This module is independent and can be reused or modified without affecting other parts of the program.

```
def subtract(a, b):
    return a - b
```

The subtract() function is another independent module. It handles subtraction and is called by the main module when needed.

```
def multiply(a, b):
    return a * b
```

The multiply() function is a self-contained module responsible for multiplication. Like the other modules, it is reusable and can be tested independently.

```
def divide(a, b):
    if b == 0:
        return "Error: Division by zero"
    return a / b
```

The divide() function is a modular component that handles division. It includes additional logic to prevent errors when dividing by zero.

Design Concepts – 5. Modularity – Calculator Example

- **Separation of Concerns :** Each operation (addition, subtraction, multiplication, division) is implemented in its own module. This ensures that each module has a single responsibility.
- **Reusability :** Each module can be reused in other parts of the program or even in different projects.
- **Maintainability :** If you need to fix or improve a specific operation, you only need to modify the corresponding module. For example, if you want to add more robust error handling to division, you only need to update the divide() function.

Design Concepts – 5. Modularity – Calculator Example

- Scalability : Adding new features (e.g., exponentiation, square root) is easy because you can create new modules without affecting existing ones. For example, you could add a new power() function and integrate it into the main() module.
- Testability : Each module can be tested independently. For example, you can write unit tests for the add() function without needing to run the entire program.

Design Concepts – 5. Modularity – Key benefits

- Easier to understand – Breaking down a system into smaller parts makes it more manageable.
- Improves reusability – Modules can be reused in different projects.
- Simplifies debugging & maintenance – Issues can be fixed within individual modules without affecting the entire system.
- Enhances scalability – Adding new features is easier when software is modular.
- Encourages parallel development – Different teams can work on different modules simultaneously.

Design Concepts – 5. Modularity – Criteria's

Criteria for Evaluating Modularity	Explanation
Modular Decomposability	▪ The system should be systematically divided into subproblems. ▪ Helps in problem-solving by breaking it into smaller, more manageable tasks. ▪ Example: A banking system can be divided into modules like Account Management, Loan Processing, and Transactions.
Modular Composability	▪ Existing modules should be reusable in new systems. ▪ Reduces development time and improves efficiency. ▪ Example: A payment gateway module (e.g., PayPal API) can be reused across multiple e-commerce platforms.

Design Concepts – 5. Modularity – Criterias

Criteria for Evaluating Modularity	Explanation
Modular Understandability	▪ Each module should be understandable as a standalone unit without referring to other modules. ▪ Makes debugging and testing easier. ▪ Example: A User Authentication Module (Login, Logout, Password Reset) should function independently of the Shopping Cart Module in an e-commerce app.
Modular Continuity	▪ Small changes in system requirements should only affect individual modules, not the entire system. ▪ Enhances flexibility and makes modification easier. ▪ Example: If the tax calculation rules change in an invoicing system, the update should only affect the Tax Module, not the entire billing system.

Design Concepts – 5. Modularity – Criterias

Criteria for Evaluating Modularity	Explanation
Modular Protection	▪ If an unexpected error occurs in one module, its effects should not impact other modules. ▪ Improves system stability and security. ▪ Example: In a flight booking system, if the seat selection module crashes, it should not affect the payment module.

Design Concepts – 5. Modularity – Balancing Act

- **Undermodularity** – Too few modules make the software too complex and difficult to manage (like a single large monolithic program).
- **Overmodularity** – Too many small modules increase integration complexity and cost.
- There is an optimal number (M) of modules that balances development cost and system manageability.

Design Concepts – 6. Component Design - Cohesion

- Cohesion refers to the degree to which the elements inside a module belong together.
- A highly cohesive module performs a single, well-defined task with minimal dependency on other modules.
- Higher cohesion is desirable because it makes software easier to maintain, test, and extend.
- Lower cohesion indicates randomly grouped functionalities, making the system harder to understand and modify.

Design Concepts – 6. Component Design - Cohesion

- Cohesion is a fundamental concept in software design that refers to how closely the elements (functions, data, etc.) within a module are related to each other.
- High cohesion means that the elements inside a module are strongly related and work together to achieve a single, well-defined task.
- Low cohesion, on the other hand, indicates that the elements are loosely related or unrelated, making the module harder to understand, maintain, and extend.

Design Concepts – 6. Component Design - Cohesion

Cohesion Type	Explanation	Example	Problem
Coincidental Cohesion (Lowest, Undesirable)	▪ Unrelated functions are grouped together randomly in a module. ▪ Happens due to poor design without proper planning.	A module that prints a report, clears the screen, and plays music—these tasks have no meaningful connection.	Difficult to maintain and modify, as it lacks logical grouping.
Logical Cohesion	▪ Functions that perform logically similar operations are grouped into a module, but the exact function executed depends on external control.	A module that handles all output operations but determines at runtime whether to print on a screen, send to a printer, or write to a file.	Not optimal because module functionality depends on external conditions.

Design Concepts – 6. Component Design - Cohesion

Cohesion Type	Explanation	Example	Problem
Temporal Cohesion	▪ Functions are grouped together because they execute at the same time, not because they are related in function.	A module that initializes variables, opens files, and clears the screen at program startup.	If one task changes, unrelated tasks may be affected, reducing maintainability.
Communication Cohesion	▪ Functions operate on the same data and are grouped into a module.	A module that retrieves temperature data, logs it to a file, and displays it on the screen.	Improves modularity but still ties functions together due to shared data.
Functional Cohesion (Highest, Most Desirable)	▪ The module performs a single, well-defined task.	A module that calculates the average of a list of numbers.	Benefit: Easy to maintain, test, and extend, as it focuses on one responsibility.

Design Concepts – 6. Component Design - Cohesion

➢Coincidental Cohesion (Lowest, Undesirable)

- ➢Functions or elements are grouped together randomly without any meaningful relationship
- ➢A module that performs three unrelated tasks: printing a report, clearing the screen, and playing music
- ➢Issue - These tasks have no logical connection, making the module hard to maintain and modify. If you need to change how the music is played, you might accidentally affect the report printing logic.

```
def random_module():
    print_report()  # Task 1: Print a report
    clear_screen()  # Task 2: Clear the screen
    play_music()    # Task 3: Play music
```

Design Concepts – 6. Component Design - Cohesion

➢Logical Cohesion

- ➢Functions that perform similar operations are grouped together, but the exact function executed depends on external control (e.g., parameters or flags).
- ➢A module that handles all output operations, such as printing to the screen, sending to a printer, or writing to a file. The specific action depends on the input.
- ➢Issue - While the functions are logically similar (all related to output), the module's behavior depends on external conditions, which can make it harder to test and maintain.

```
def output_handler(output_type, data):
    if output_type == "screen":
        print_to_screen(data)
    elif output_type == "printer":
        send_to_printer(data)
    elif output_type == "file":
        write_to_file(data)
```

Design Concepts – 6. Component Design - Cohesion

➢Temporal Cohesion

- ➢Functions are grouped together because they execute at the same time, not because they are functionally related..
- ➢A module that initializes variables, opens files, and clears the screen at program startup.
- ➢Issue - These tasks happen at the same time (program startup), but they are not functionally related. If one task changes, it could inadvertently affect unrelated tasks, reducing maintainability.

```
def startup_module():
    initialize_variables()  # Task 1: Initialize variables
    open_files()            # Task 2: Open necessary files
    clear_screen()          # Task 3: Clear the screen
```

Design Concepts – 6. Component Design - Cohesion

➢Communication Cohesion

- ➢Functions operate on the same data and are grouped into a module
- ➢A module that retrieves temperature data, logs it to a file, and displays it on the screen.
- ➢Merits - The module is more cohesive because all functions operate on the same data (temperature). However, the functions are still tied together due to shared data, which can make it slightly harder to modify individual parts.

```
def temperature_module():
    temp = get_temperature()       # Task 1:
Retrieve temperature
    log_temperature(temp)          # Task 2:
Log temperature to a file
    display_temperature(temp)      # Task 3:
Display temperature on the screen
```

Design Concepts – 6. Component Design - Cohesion

- **Functional Cohesion (Highest, Most Desirable)**
  - The module performs a single, well-defined task
  - A module that calculates the average of a list of numbers.
  - Merits - This module has high cohesion because it focuses on one responsibility: calculating the average. It is easy to maintain, test, and extend because it does one thing and does it well.

```
def calculate_average(numbers):
    total = sum(numbers)
    count = len(numbers)
    return total / count
```

Design Concepts – 6. Component Design - Cohesion

Types of Cohesion	Description	Example
Coincidental	Random grouping of unrelated functions	Printing a report, clearing the screen, and playing music
Logical	Grouping of logically similar functions	Handling all output operations (screen, printer, file)
Temporal	Grouping of functions that execute at the same time	Initializing variables, opening files, and clearing the screen at startup
Communication	Grouping of functions that operate on the same data	Retrieving, logging, and displaying temperature data
Functional	Module performs a single, well-defined task	Calculating the average of a list of numbers

Design Concepts – 6. Component Design - Cohesion

➢Why Higher Cohesion is Desirable:

- ➢Easier to Maintain : Modules with high cohesion are easier to understand and modify because they focus on a single task.
- ➢Easier to Test : Testing is simpler because each module has a clear responsibility.
- ➢Easier to Extend : Adding new features or modifying existing ones is less risky because modules are self-contained and focused.

Design Concepts – 7. Component Design - Coupling

- Coupling refers to the degree of dependency between different software modules or components.

- Lower coupling is desirable because it makes software easier to modify, test, and maintain.

- High coupling should be avoided as it increases system complexity and reduces flexibility.

Design Concepts – 7. Component Design - Coupling

- Coupling refers to the degree of interdependence between different modules or components in a software system.
- In other words, it measures how closely connected one module is to another.
- Lower coupling is desirable because it makes the system more modular, easier to maintain, test, and extend.
- On the other hand, high coupling increases complexity, reduces flexibility, and makes the system harder to modify.

Design Concepts – 7. Component Design - Coupling

Coupling Type	Explanation	Example	Problem
Content Coupling (Strongest, Most Undesirable)	▪ One module modifies the internal data or logic of another module.	Module A directly changes a variable inside Module B.	Violates information hiding and makes debugging and maintenance difficult.
Common Coupling	▪ Multiple modules share the same global data.	Many modules access and modify a global configuration variable.	Changes in one module may cause unexpected errors in others.
External Coupling	▪ Modules depend on external systems such as databases, files, or network services.	A web application module depends on a third-party API for authentication.	The system becomes vulnerable to external changes or failures.

Design Concepts – 7. Component Design - Coupling

Coupling Type	Explanation	Example	Problem
Control Coupling	▪ One module controls another module's behavior by passing control flags or parameters.	Function A calls function B and passes a flag to decide what function B should do.	If function B changes, function A may also need modification, making maintenance harder.
Stamp Coupling	▪ Modules pass complex data structures instead of specific required data.	A function that sends an entire student object to another function, even if only the student's ID is needed.	Unnecessary dependencies are created, making changes more complex.
Data Coupling (Weakest, Most Desirable)	▪ Modules share only the necessary data in function calls.	A function passes only required values like (name, age) instead of the full user object.	Easier maintenance and better modularity.

Design Concepts – 7. Component Design - Coupling

- Content Coupling (Strongest, Most Undesirable)
  - One module directly modifies or depends on the internal data or logic of another module.
  - Module A directly accesses and changes a variable inside Module B.
  - This violates encapsulation and information hiding. If Module B changes its internal structure, Module A will break. Debugging and maintaining such code becomes difficult because the internal details of Module B are exposed to Module A.

```
# Module B
class ModuleB:
    def __init__(self):
        self.internal_data = 0

# Module A
def modify_module_b(module_b_instance):
    module_b_instance.internal_data = 42  # Directly modifying internal data of Module B
```

Design Concepts – 7. Component Design - Coupling

- Content Coupling (Strongest, Most Undesirable)
  - One module directly modifies or depends on the internal data or logic of another module.
  - Module A directly accesses and changes a variable inside Module B.
  - This violates encapsulation and information hiding. If Module B changes its internal structure, Module A will break. Debugging and maintaining such code becomes difficult because the internal details of Module B are exposed to Module A.

```
# Module B
class ModuleB:
    def __init__(self):
        self.internal_data = 0

# Module A
def modify_module_b(module_b_instance):
    module_b_instance.internal_data = 42  # Directly modifying internal data of Module B
```

Design Concepts – 7. Component Design - Coupling

- Common Coupling
  - Multiple modules share the same global data, leading to dependencies on that shared data.
  - Several modules access and modify a global configuration variable..
  - Changes to the global variable by one module can cause unexpected behavior in other modules. For example, if Module A changes the theme, Module B might behave differently without any direct interaction between them. This creates hidden dependencies and makes the system harder to debug.

```
# Global variable
global_config = {"theme": "dark", "language":
"en"}

# Module A
def change_theme(new_theme):
    global_config["theme"] = new_theme

# Module B
def get_current_theme():
    return global_config["theme"]
```

Design Concepts – 7. Component Design - Coupling

- External Coupling
  - Modules depend on external systems, such as databases, files, or network services.
  - A web application module depends on a third-party API for authentication.
  - The system becomes vulnerable to external changes or failures. If the third-party API changes its interface or goes down, the entire system may fail. This type of coupling reduces the system's flexibility and reliability.

```
import requests

def authenticate_user(username, password):
    response =
requests.post("https://api.example.com/auth",
data={"username": username, "password":
password})
    return response.json()
```

Design Concepts – 7. Component Design - Coupling

- Control Coupling
  - One module controls the behavior of another module by passing control flags or parameters.
  - Function A calls function B and passes a flag to decide what function B should do..
  - If function_b changes its internal logic or the meaning of the control flag, function_a may also need modification. This creates tight coupling between the two functions, making maintenance harder..

```
def function_b(action):
    if action == "start":
        print("Starting process...")
    elif action == "stop":
        print("Stopping process...")

def function_a():
    function_b("start")  # Controlling the behavior of function_b
```

Design Concepts – 7. Component Design - Coupling

- Stamp Coupling
  - Modules pass complex data structures (e.g., objects or records) instead of only the specific data needed.
  - A function sends an entire student object to another function, even though only the student's ID is required
  - The process_student function only needs the student's ID, but it receives the entire student object. This creates unnecessary dependencies, making the system harder to modify. If the Student class changes, process_student may also need to be updated...

```
class Student:
    def __init__(self, id, name, age):
        self.id = id
        self.name = name
        self.age = age

def process_student(student):
    print(f"Processing student with ID:
{student.id}")

def main():
    student = Student(1, "Alice", 20)
    process_student(student)  # Passing the
entire student object
```

Design Concepts – 7. Component Design - Coupling

- Data Coupling
  - Modules share only the necessary data through function calls, without exposing unnecessary details..
  - A function passes only the required values (e.g., name and age) instead of the full user object.
  - This is the weakest form of coupling and the most desirable. Each module is independent and only shares the minimal amount of data required. It makes the system easier to maintain, test, and extend because there are no unnecessary dependencies.

```
def greet_user(name, age):
    print(f"Hello, {name}! You are {age} years old.")

def main():
    greet_user("Alice", 20)  # Passing only the
necessary data
```

Design Concepts – 7. Component Design - Coupling

Types of Coupling	Description	Example
Content Coupling	One module modifies the internal data or logic of another module	Module A directly changes a variable inside Module B
Common Coupling	Multiple modules share the same global data	Many modules access and modify a global configuration variable
External Coupling	Modules depend on external systems like databases or APIs	A web application depends on a third-party API for authentication
Control Coupling	One module controls another module's behavior by passing control flags	Function A passes a flag to decide what Function B should do
Stamp Coupling	Modules pass complex data structures instead of specific required data	A function receives an entire student object when only the ID is needed
Data Coupling	Modules share only the necessary data in function calls	A function passes only required values like (name, age)

Design Concepts – 8. Refinement

- Refinement is a stepwise approach where a complex system or problem is gradually broken down into smaller, more manageable components.

- It is a top-down strategy, also known as Stepwise Refinement.

- The goal is to refine procedural details until they can be implemented in a programming language.

Design Concepts – 8. Refinement

- Start from an abstract level and refine it step by step.
- Move from high-level concepts to detailed procedural steps.
- This ensures systematic decomposition rather than jumping into low-level coding immediately.

- Example:
  - Begin with a high-level algorithm for sorting.
  - Refine it into specific steps (e.g., divide, compare, swap).
  - Finally, implement it in actual code (e.g., QuickSort, MergeSort).

Design Concepts – 8. Refinement

- A system is not designed all at once; instead, details are added progressively.
- Each level of refinement adds more specificity to a function or process.
- Ensures that the design is well-structured and understandable.

Design Concepts – 8. Refinement

- Example
  - Consider designing a login system:
  - High-Level Process: Authenticate user.
    - Refined Steps:
      - Prompt user for username and password.
      - Validate user input.
      - Check credentials in the database.
      - Grant or deny access.
  - Final Implementation:
    - Use encryption for passwords.
    - Implement OAuth authentication if required.

Design Concepts – 8. Refinement

- Refinement helps in breaking down a large process into smaller, self-contained functions.
- Each step is further decomposed until it can be coded directly.
- Hierarchy is formed, with abstract steps at the top and detailed implementation at the bottom.

Design Concepts – 8. Refinement

➢Developing a **file processing system**:

- **High-Level Task:** Process a file.
- **Refinement Steps:**
  - Open the file.
  - Read the content.
  - Process the data.
  - Save the results.
- **Further Breakdown:**
  - "Read the content" → Read line by line, extract metadata, etc.
  - "Process the data" → Apply data transformation, validation, filtering.

Design Concepts – 8. Refinement

- Refinement in design & coding is like partitioning in requirements analysis.
- In requirement analysis, high-level requirements are partitioned into functional modules.
- In design & coding, high-level procedures are refined into detailed implementations.

Design Concepts – 8. Architecture Design - Transform

- Architectural Mapping Using Data Flow
  - Helps transition from data flow diagrams (DFDs) to software architecture.
  - **Six-step process:**
  - Identify Information Flow Type – Determine if the system has a transform or transaction flow.
  - Define Flow Boundaries – Identify where data enters and exits the system.
  - Map DFD to Program Structure – Convert DFDs into an organized software architecture.
  - Define Control Hierarchy – Establish the decision-making structure.
  - Refine Design Using Heuristics – Optimize for functional independence, cohesion, and minimal coupling.
  - Elaborate Architectural Description – Improve and document architecture.

Design Concepts – 8. Architecture Design - Transform

- In the architectural mapping process, two major types of information flow play a critical role in detailed design: Transform Mapping and Transaction Mapping.

- These approaches help transition from the data flow diagrams (DFDs) used in the requirements model to a structured software architecture.

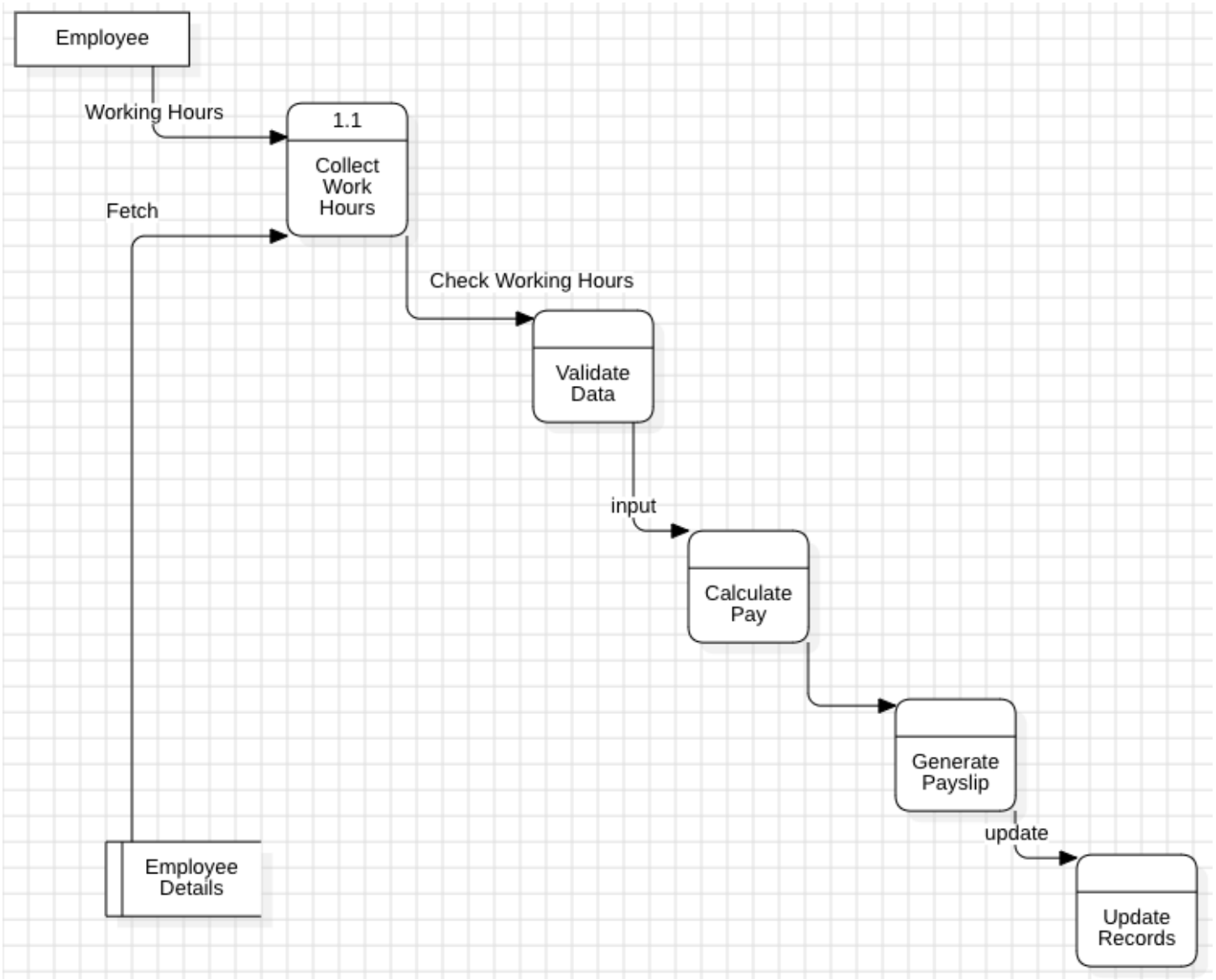
Design Concepts – 8. Transform Mapping

- Transform mapping applies when the system exhibits a transform flow, where data flows through a series of processing steps in a well-defined sequence.
- **Characteristics of Transform Flow**
  - Linear flow of data processing.
  - Input data is transformed into an internal representation.
  - Data is processed at a transform center (core processing logic).
  - Output data is then converted back to an external representation.

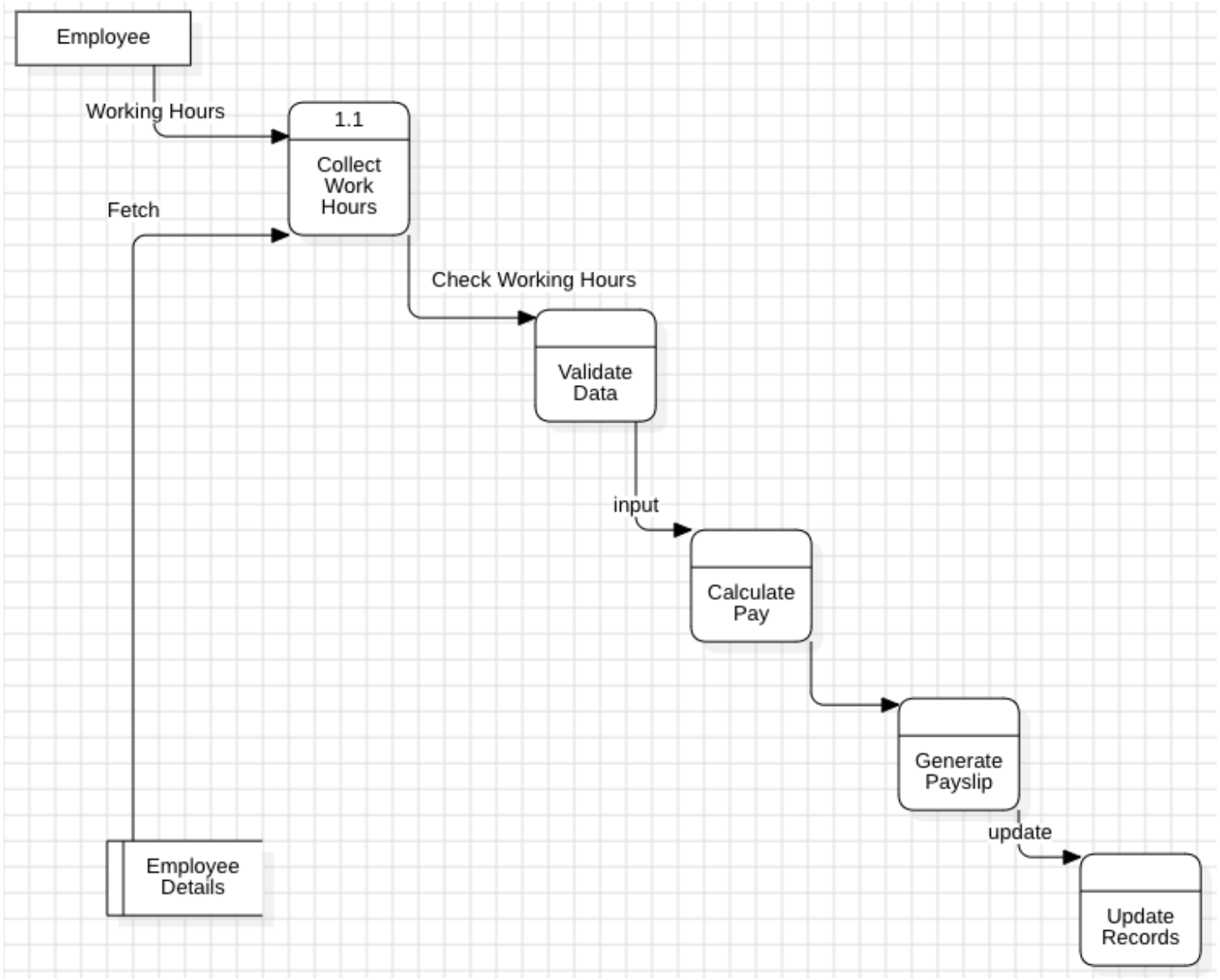
Design Concepts – 8. Transform Mapping

- **Steps in Transform Mapping**
  - Identify Information Flow – Determine if the DFD exhibits transform flow.
  - Define Flow Boundaries – Identify where the system transitions between external and internal data.
  - Identify Transform Center – The core processing stage of the flow.
  - Perform First-Level Factoring – Organize the architecture in a top-down manner:
    - Top-Level Components handle decision-making.
    - Middle-Level Components manage control logic.
    - Low-Level Components perform data input, computation, and output.
  - Second-Level Factoring – Map individual DFD transformations into software modules.
  - Refine Architecture – Improve cohesion, reduce coupling, and enhance maintainability.

Design Concepts – 8. Transform Mapping - Example

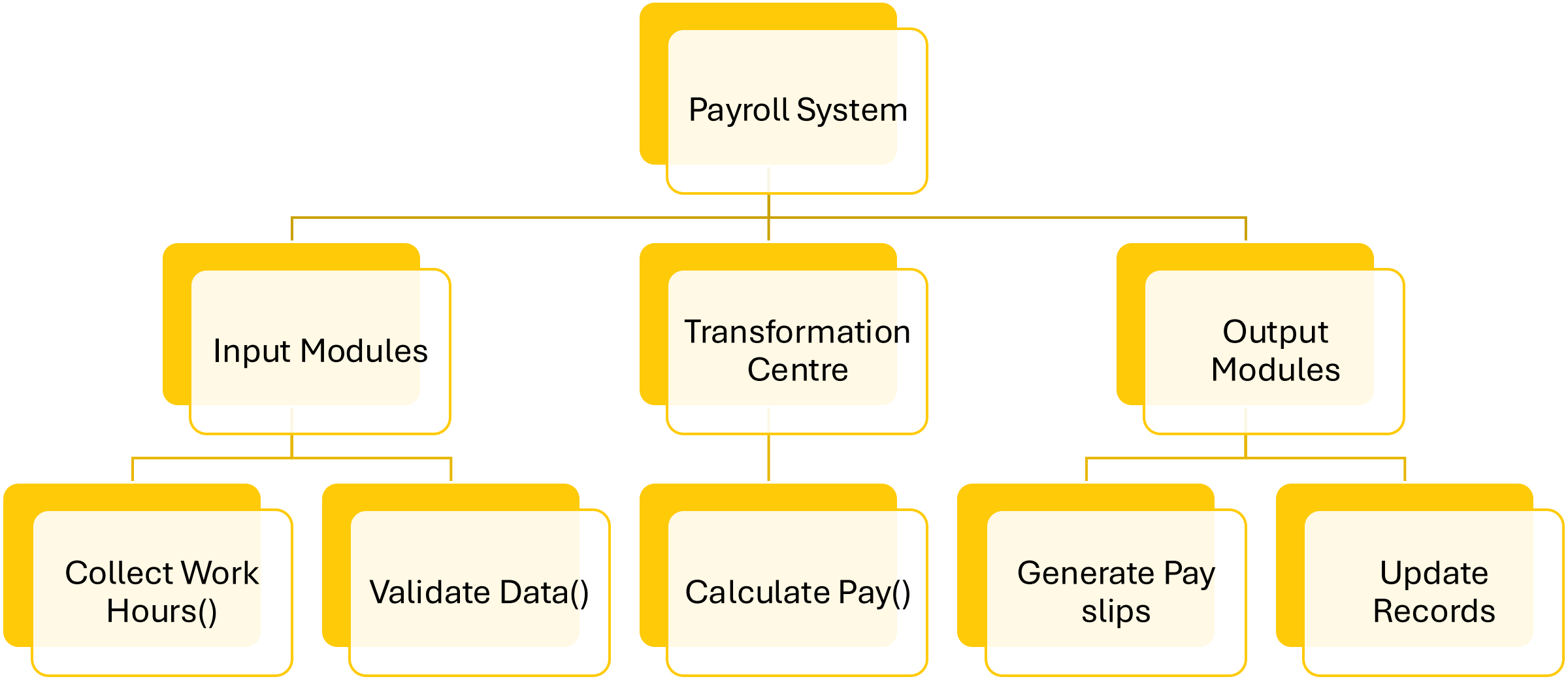


Design Concepts – 8. Transform Mapping - Example



Input Domain	Transform Centre	Output Domain
Collect Work Hours	Calculate Pay	Generate Pay slips
Validate Data		Update Records

Design Concepts – 8. Transform Mapping - Example



Design Concepts – 8. Transaction Mapping

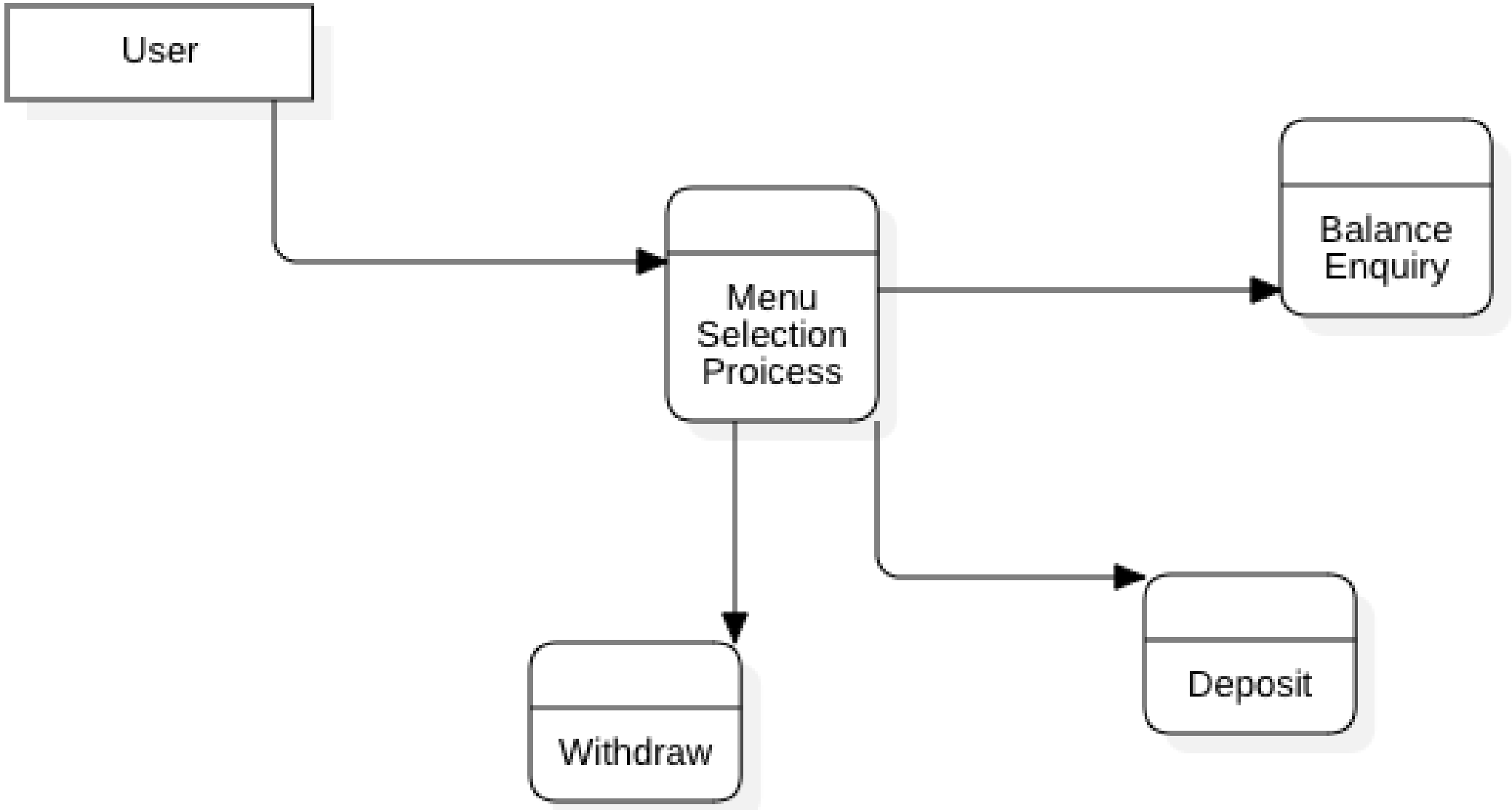
- Transaction mapping applies when the system exhibits a transaction flow, where multiple possible execution paths exist based on user or system actions.

- **Characteristics of Transaction Flow**
  - System behaviour is triggered by an external transaction (e.g., user input, an event).
  - The system branches into different execution paths based on the type of transaction.
  - A transaction centre controls decision making and routes requests accordingly.

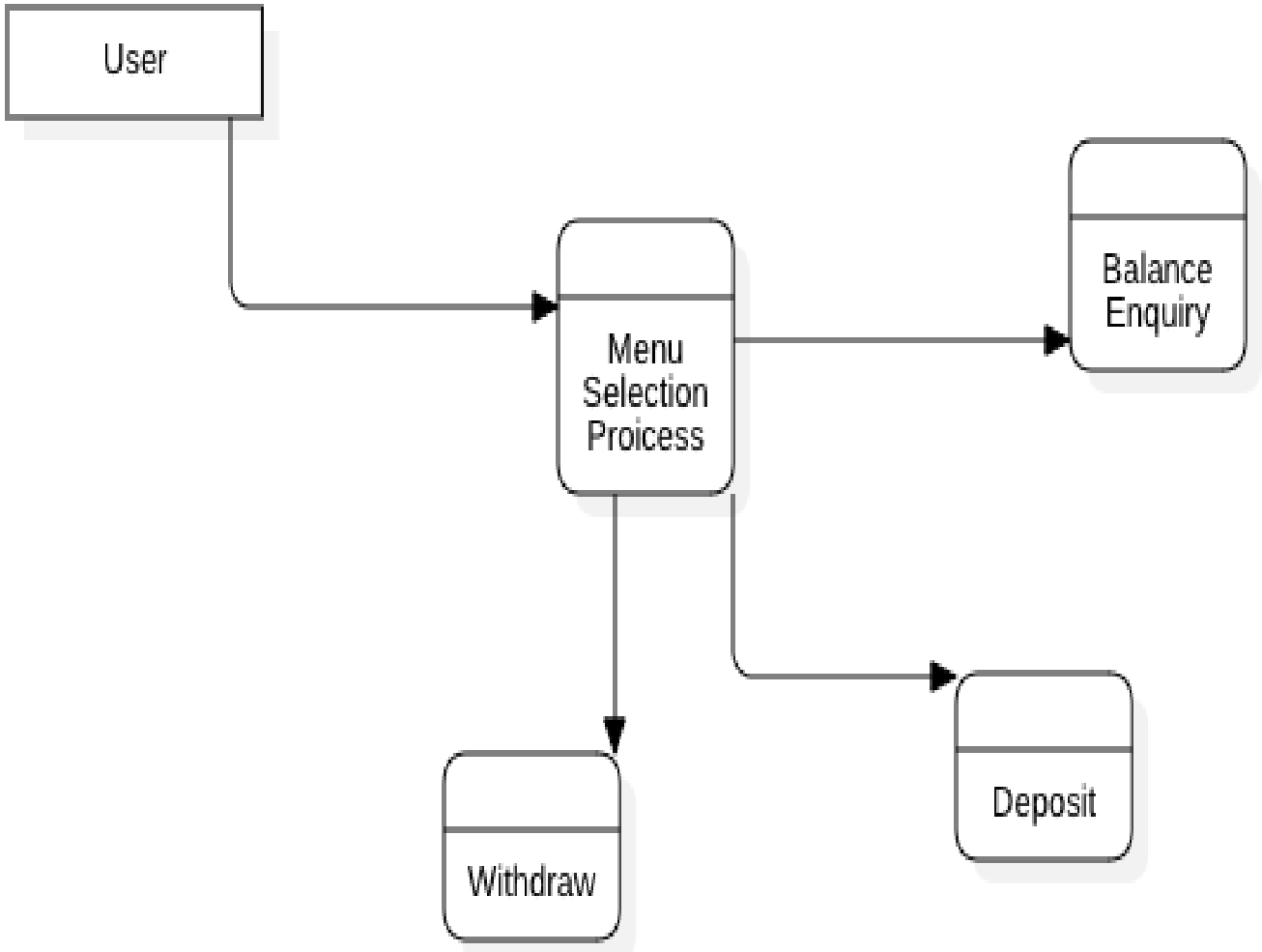
Design Concepts – 8. Transaction Mapping

- **Steps in Transaction Mapping**
  - **Identify the Transaction Center** – The module that controls branching (e.g., a request handler).
  - **Define Input Transactions** – Identify external triggers that activate different processing paths.
  - **Establish Processing Paths** – Each transaction type has a unique execution flow.
  - **Map DFDs to Program Structure** – Assign modules to handle each type of transaction.
  - **Refine Software Architecture** – Optimize for independence, modularity, and scalability.

Design Concepts – 8. Transaction Mapping - Example



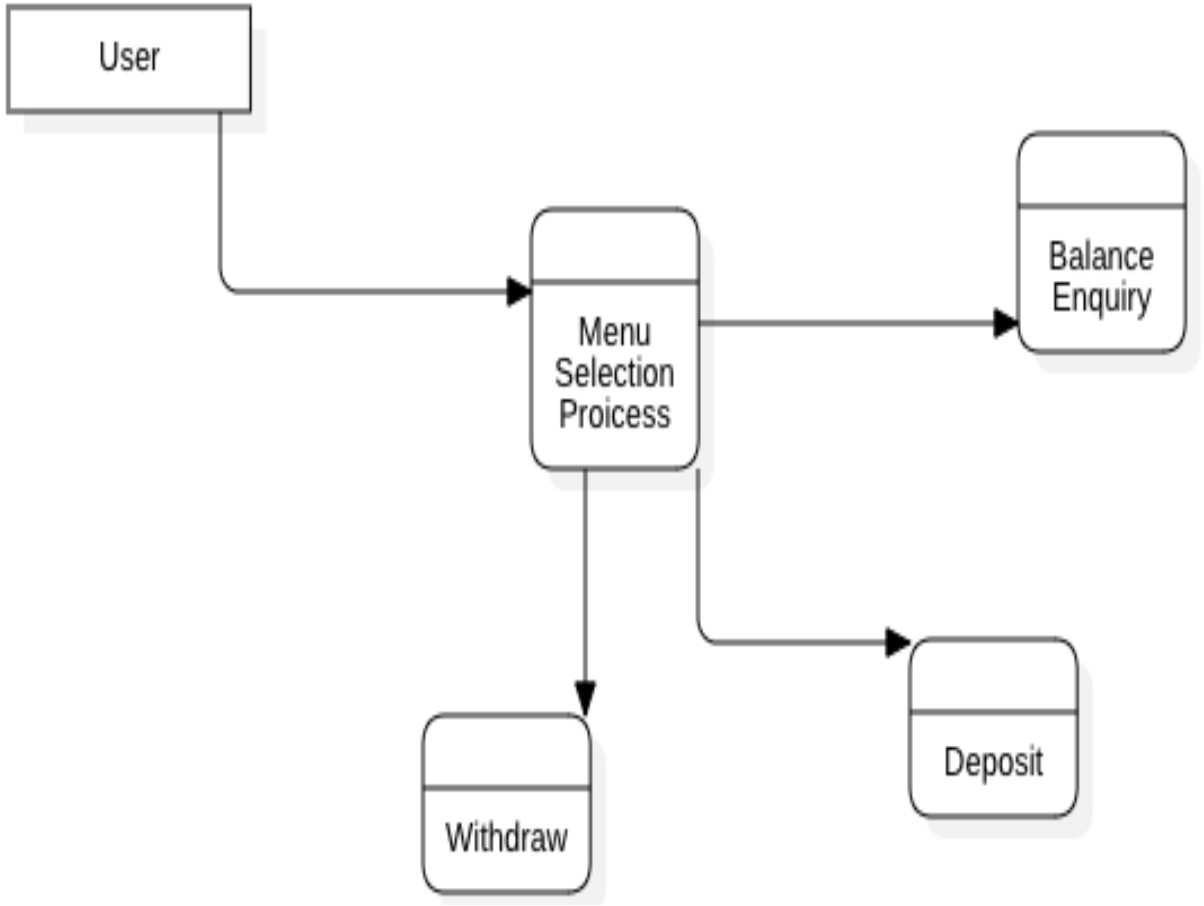
Design Concepts – 8. Transaction Mapping - Example

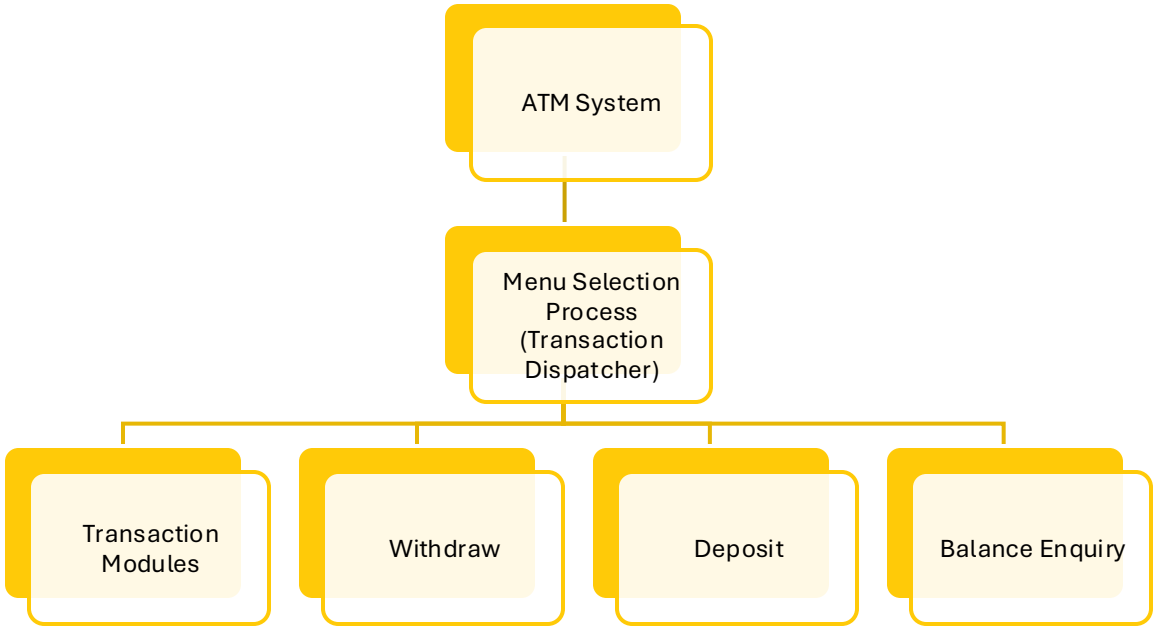


Transaction Center: "Menu Selection Process"

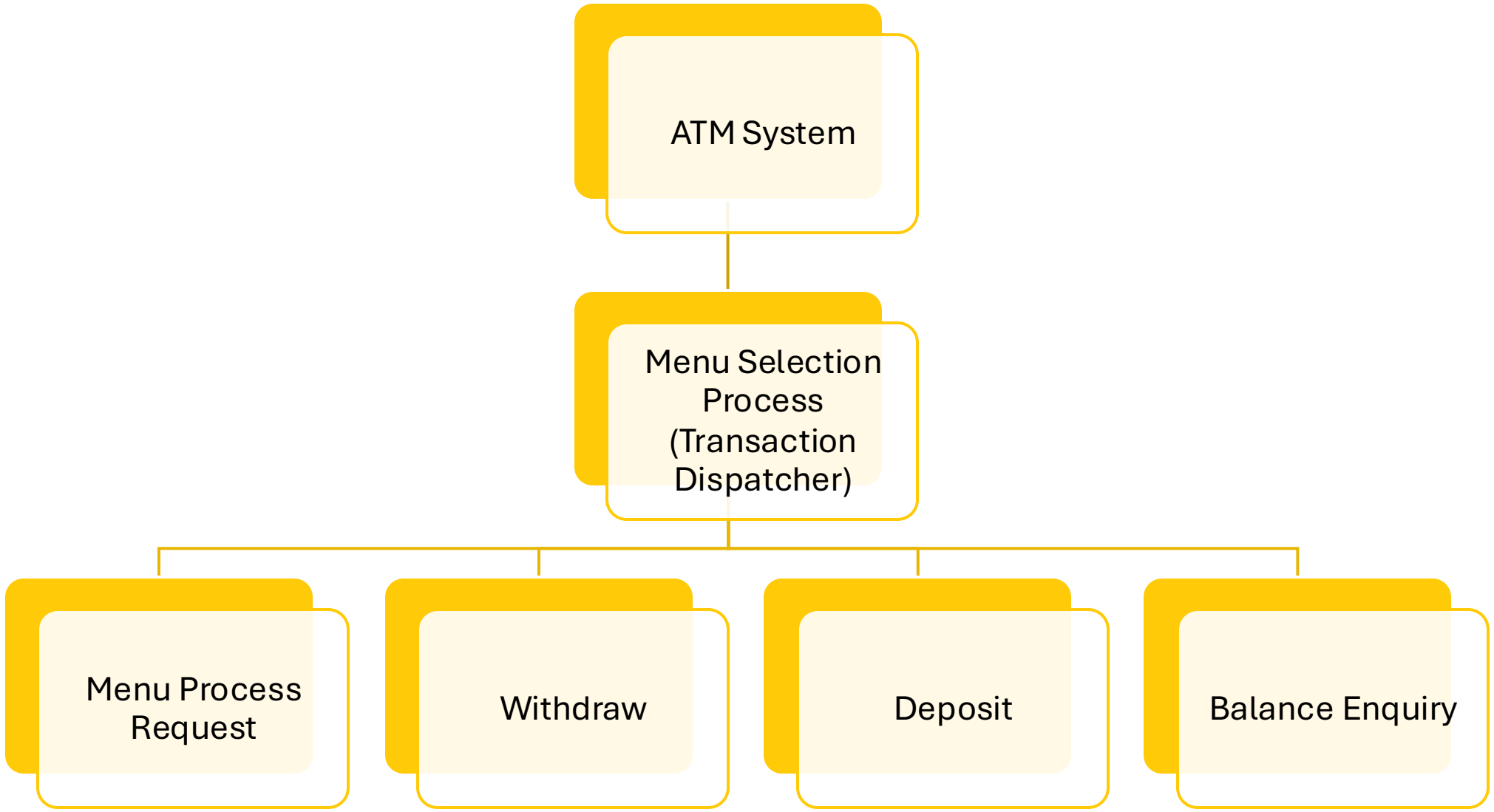
The system **branches into different transactions** (Withdraw, Deposit, Balance Inquiry) based on user selection.

Design Concepts – 8. Transaction Mapping - Example





Design Concepts – 8. Transaction Mapping - Example



Design Concepts – 9. Refactoring

- Refactoring is a reorganization technique that improves software design or code without changing its functionality or behavior.

- Martin Fowler defines refactoring as modifying a software system to enhance its internal structure while maintaining external behavior.

Design Concepts – 9. Refactoring

- **Purpose of Refactoring**
  - Eliminates redundancy and unused design elements.
  - Improves efficiency by optimizing algorithms and data structures.
  - Enhances readability, maintainability, and scalability of the software.

- **Identifying the Need for Refactoring**
  - Detects poorly constructed or inappropriate design elements.
  - Identifies low cohesion components performing unrelated functions.
  - Recognizes unnecessary complexities in the system design.

Design Concepts – 9. Refactoring

- **Benefits of Refactoring**
  - Leads to higher cohesion by breaking down complex components into smaller, well-defined modules.
  - Results in software that is easier to integrate, test, and maintain.
  - Reduces technical debt, making future modifications smoother.
- **Example of Refactoring**
  - A component performing three loosely related functions is refactored into three separate components with high cohesion.
  - This improves modularity, simplifies testing, and enhances code maintainability.

User Interface Design – Golden Rules

Place the User in Control

Allow user control over the system	• Design interfaces that react to user needs, enabling users to feel in charge rather than controlled by the system.
Define interaction modes flexibly	• Avoid forcing users into unnecessary or undesired actions; allow easy entry and exit from modes (e.g., spell-check mode).
Provide flexible interaction options	• Support multiple interaction methods (keyboard, mouse, touch, voice) to accommodate diverse user preferences.
Enable interruptible and undoable actions	• Allow users to pause sequences, switch tasks, and undo actions without losing progress.
Streamline interaction for advanced users	• Offer customization features like macros to simplify repetitive tasks for skilled users.
Hide technical details	• Shield casual users from system internals (e.g., operating system commands) and focus on task-oriented interactions.
Enable direct manipulation	• Allow users to interact directly with on-screen objects (e.g., resizing or moving items) for a sense of control.

User Interface Design – Golden Rules

Reduce the User's Memory Load

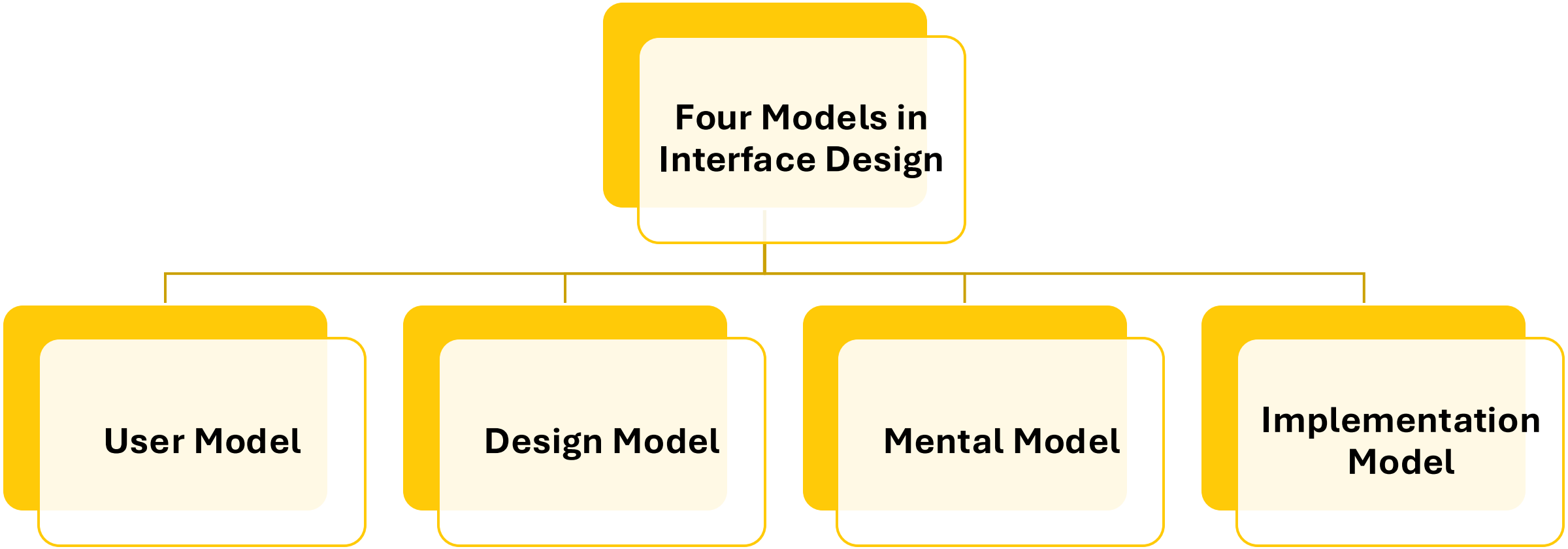
Minimize reliance on short-term memory	• Use visual cues to help users recognize past actions instead of recalling them.
Set meaningful defaults	• Provide sensible default settings for average users while allowing personalization and offering a reset option.
Design intuitive shortcuts	• Use mnemonics tied to actions (e.g., alt-P for print) that are easy to remember.
Use real-world metaphors	• Base interface layouts on familiar concepts (e.g., checkbook for bill payments) to reduce the need for memorization.
Progressive disclosure of information	• Present high-level overviews first and reveal details progressively as users show interest (e.g., hierarchical menus).

User Interface Design – Golden Rules

Make the Interface Consistent

Organize visual information consistently	• Apply uniform design rules across all screens to ensure clarity and coherence.
Limit and standardize input mechanisms	• Use a consistent set of input methods throughout the application to avoid confusion.
Standardize navigation	• Define and implement consistent ways to move between tasks.
Provide meaningful context	• Use indicators (titles, icons, colors) to help users understand their current task and navigate effectively.
Maintain consistency across applications	• Ensure all products in a family follow the same design principles for a unified experience.
Respect established user expectations	• Avoid changing widely accepted interaction patterns (e.g., alt-S for save) unless necessary, as changes can confuse users.

User Interface Design – Models

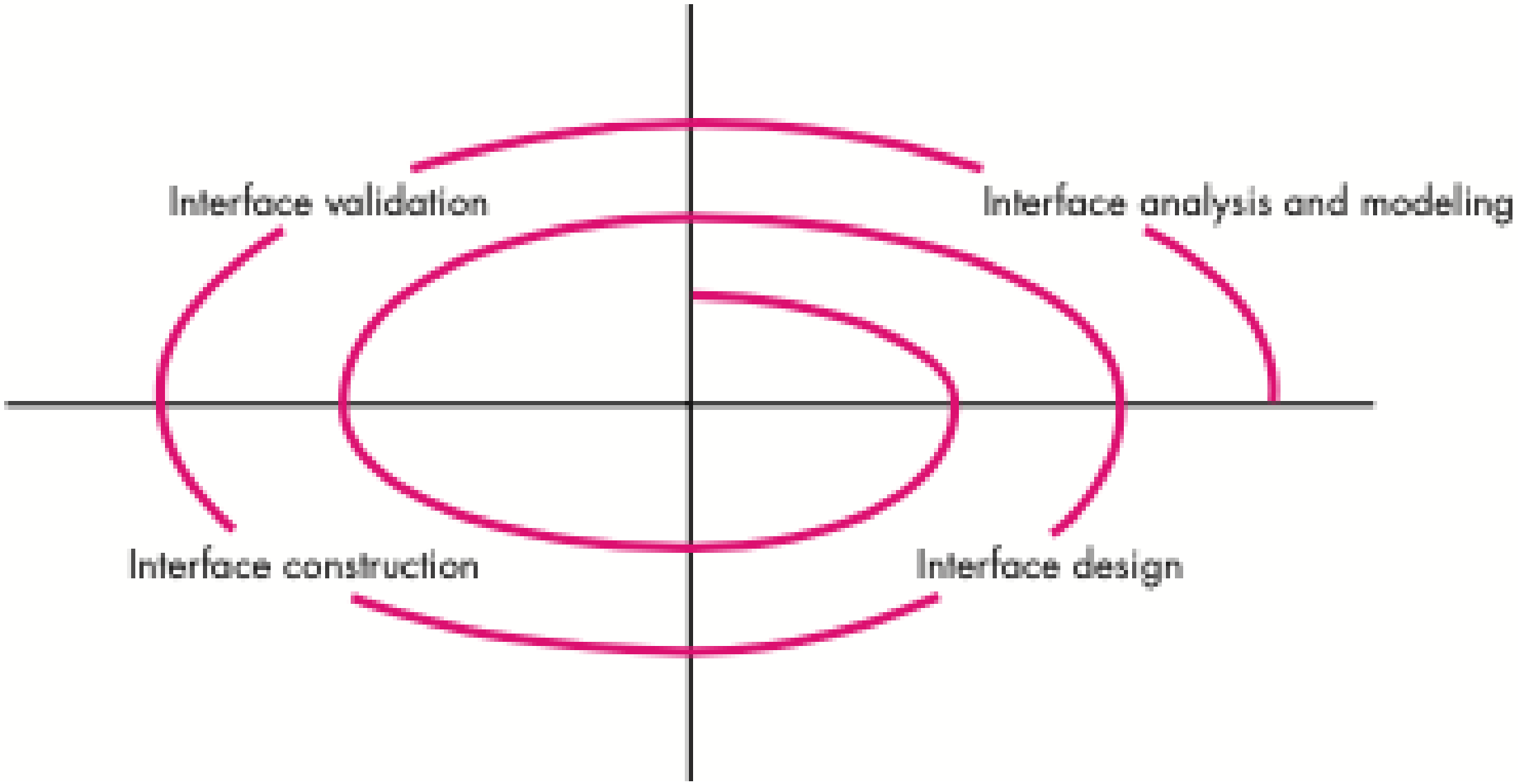


User Interface Design – Models

User Model	• Establishes the profile of end users, including demographics, abilities, and goals.
	• Users can be categorized as:
	  • **Novices** : No syntactic or semantic knowledge of the system.
	  • **Knowledgeable, Intermittent Users** : Understand the application but struggle with interface details.
	  • **Knowledgeable, Frequent Users** : Skilled users who seek shortcuts and efficient interactions.

Design Model	• Created by software engineers to align with the user model and guide the design process.
Mental Model (System Perception)	• The user's internal understanding of how the system works, influenced by their experience and familiarity.
Implementation Model	• Combines the system's outward appearance (look and feel) with supporting documentation (manuals, help files). • Goal - Align the implementation model with the user's mental model for effective and comfortable use.
Key Principle	• Know the user, know the tasks." Designers must avoid substituting their own perspective for that of the user.

User Interface Design – Process



User Interface Design – Process

- The user interface design process is **iterative**, represented by a spiral model with four framework activities:
- **Interface Analysis and Modeling**
  - Focus on understanding user profiles, skill levels, and environmental factors.
  - Define user categories and elicit requirements for each category.
  - Conduct task analysis to identify and describe user tasks.
  - Analyze the physical work environment (e.g., location, lighting, noise, human factors).
- **Interface Design**
  - Define interface objects, actions, and screen representations to meet usability goals.
  - Ensure tasks can be performed efficiently and intuitively.
- **Interface Construction**
  - Begin with a prototype to evaluate usage scenarios.
  - Use user interface toolkits to refine and complete the interface.
- **Interface Validation**
  - Validate that the interface supports all user tasks and variations.
  - Assess ease of use, learnability, and user acceptance.

User Interface Design – Process

- **Prototyping :**
  - Prototyping is essential for evaluating and refining the interface during construction.

- **Iterative Nature :**
  - The process involves multiple passes through the spiral, elaborating task details, design information, and operational features incrementally.
  - There is no need to specify every detail in the first iteration; subsequent iterations refine the design.

User Interface Design – Design Issues

System Response Time

- Measured from user action to system response.
- Two key characteristics:
  - Length : Long delays cause frustration.
  - Variability : Consistent response times are preferable to highly variable ones, even if slightly longer.
  - Goal: Minimize variability to establish a predictable interaction rhythm.

Help Facilities

- Essential for guiding users when they encounter difficulties.
- Key design considerations:
  - Availability: Provide help for all functions or a subset.
  - Access: Offer help via menus, function keys, or commands.
  - Representation: Use separate windows, one-line suggestions, or hypertext.
  - Structure: Organize help as flat, layered, or hypertext-based.
  - Return: Allow easy return to normal interaction.

User Interface Design – Design Issues

Error Handling

Error messages should be clear, constructive, and nonjudgmental.

Characteristics of effective error messages:

1. Use understandable language.
2. Provide actionable advice for recovery.
3. Highlight negative consequences (e.g., data corruption).
4. Include visual or auditory cues (e.g., color, beep).
5. Avoid blaming the user.

Poorly designed error messages increase user frustration and confusion.

User Interface Design – Design Issues

Menu and Command Labelling

1. Ensure consistency and clarity in command usage across applications.
2. Key considerations:
    1. Correspondence: Match menu options with commands.
    2. Form: Use control sequences, function keys, or typed words.
    3. Learnability: Make commands easy to learn and remember; provide reminders for forgotten commands.
    4. Customization: Allow users to customize or abbreviate commands.
    5. Clarity: Ensure menu labels and submenus are self-explanatory and consistent.

User Interface Design – Design Issues

Application Accessibility

1. Design interfaces to accommodate users with physical challenges (e.g., visual, hearing, mobility impairments).
2. Follow accessibility guidelines (e.g., W3C, assistive technology standards) to ensure inclusivity.
3. Ethical, legal, and business imperatives drive accessibility requirements.

User Interface Design – Design Issues

Internationalization

1. Create "globalized" software with a generic core functionality adaptable to different locales and languages.
2. Localization features enable customization for specific markets.
3. Challenges:
   1. Screen layouts may vary across regions.
   2. Different alphabets and symbols require specialized labeling and spacing.
4. Tools like Unicode address multilingual support challenges.

User Interface Design – characteristics

- **Visually Apparent and Forgiving :**
  - Users should feel in control, with clear options and easy recovery from mistakes.
- **Hide System Internals :**
  - Save work automatically and allow users to undo actions without worrying about system mechanics.
- **Minimize User Input :**
  - Maximize functionality while minimizing the information users need to provide.

User Interface Design – Principles

- **Anticipation :**
  - Predict user needs and provide intuitive navigation (e.g., pre-emptively offering a driver download after viewing printer information).
- **Communication :**
  - Clearly communicate activity status (e.g., text messages, animations) and user location within the WebApp.
- **Consistency :**
  - Maintain uniformity in navigation controls, menus, icons, and aesthetics throughout the interface.
  - Avoid reusing symbols or elements for conflicting purposes.
- **Controlled Autonomy :**
  - Allow users to navigate freely but enforce security and navigation conventions (e.g., password-protected areas).
- **Efficiency :**
  - Prioritize user productivity over developer convenience or system efficiency.

User Interface Design – Principles

- **Flexibility :**
  - Support both task-oriented users and exploratory users, providing undo functionality and clear navigation paths.
- **Focus :**
  - Keep the interface focused on the user's primary tasks, avoiding distractions from loosely related content.
- **Fitt's Law :**
  - Minimize the time required for users to make selections by placing related options close together.
- **Human Interface Objects :**
  - Use reusable interface objects (e.g., buttons, menus) from established libraries to enhance usability.
- **Latency Reduction :**
  - Use multitasking to reduce perceived delays, acknowledge delays with feedback (e.g., progress bars), and entertain users during long operations.

User Interface Design – Principles

- **Learnability :**
  - Design for quick learning and minimal relearning, emphasizing simple, intuitive organization of content and functionality.
- **Metaphors :**
  - Use appropriate metaphors (e.g., checkbook for bill payments) to make the interface easier to learn and use.
- **Maintain Work Product Integrity :**
  - Autosave user data to prevent loss and provide recovery mechanisms for interrupted sessions.
- **Readability :**
  - Ensure all information is readable for all age groups using clear fonts, sizes, and high-contrast colors.
- **Track State :**
  - Store user interaction states (e.g., via cookies) to allow users to resume their work later.
- **Visible Navigation :**
  - Create the illusion that users are in a single place, bringing content and functions to them rather than requiring navigation.